Tema 1

Almacenamiento de la información

Base de datos

Tech Hub <FP>

euroformac

ÍNDICE

Objetivos de la unidad 3
1. Introducción 5
2. Ficheros 7
2.1 Ficheros planos 7
2.2 Ficheros indexados 11
3. Bases de datos 14
3.1 Concepto 14
3.2 Clasificación de las bases de datos 17
3.3 Tipos de bases de datos según su estructura 18
4. Sistemas Gestores de Bases de Datos 22
4.1 Concepto 22
4.2 Tipos de SGBD 22
4.3 Objetivos y funcionalidades de los SGBD 23
4.4 Componentes de los SGBD 24
5. SGBD Comerciales vs SGBD Libres 27
5.1 SGBD Comerciales 28
5.2 SGBD Libres 29
6. Bases de datos centralizadas y distribuidas 31
6.1 Bases de datos centralizadas 31
6.2 Bases de datos distribuidas 31
7. Resumen 33
Glosario 34
Bibliografía 36

Tech Hub <FP> euroformac

Objetivos de la unidad

Los objetivos de aprendizaje de esta unidad son:

- Introducir el concepto de persistencia de la información, así como los distintos formatos en los que puede ser almacenada.
- Familiarizarnos con los distintos modelos de bases de datos existentes y cómo han evolucionado hasta la actualidad.
- Comprender qué es un sistema gestor de base de datos y cómo puede ayudarnos a gestionar las distintas tipologías de objetos que conforman una base de datos.
- Conocer el funcionamiento de las bases de datos centralizadas y las bases de datos distribuidas.

Tech Hub <FP> euroformac

Estamos acostumbrados a acceder a muchas aplicaciones y sitios web en que, tras un primer acceso, en los siguientes nos aparecen por defecto muchas de las preferencias en navegación y tipos de contenidos que nos interesaron durante nuestra primera visita.

También hemos normalizado comprar online en nuestras tiendas preferidas, proporcionando nuestros datos personales y de pago una única vez, y que cada vez que vayamos a cerrar un pedido, nos ofrezcan usarlos sin tener que volver a introducirlos de nuevo.

Podríamos hablar también de nuestras redes sociales, cuentas de correo electrónico, etc., que es donde se almacena información de distinto tipo y estructura que vamos generando en nuestro día a día y que en cualquier momento podemos consultar.

Todas estas ventajas que hemos enumerado y de las que disfrutamos, no serían posibles si no dispusiéramos de algún sistema que nos permitiera **persistir** dicha información para ser recuperada a futuro.

Y una de las formas de **persistir** la información, de una manera organizada y estructurada, es utilizando las **bases de datos** y toda su tecnología asociada.

A continuación, a través de distintas unidades didácticas, conoceremos sus particularidades y aprenderemos a diseñarlas y explotarlas de una manera óptima y eficaz para que nuestras aplicaciones puedan obtener todos los beneficios que nos proporcionan.

1. Introducción

Hoy en día, más allá de aquellas aplicaciones extremadamente sencillas, en que, a partir de unos datos introducidos a través de una interfaz, nos ofrecen un resultado que es fruto de un cálculo o un determinado proceso, necesitamos persistir (guardar, almacenar, ..) la información que registramos o generamos, para poder ser reutilizada en otro momento, ya sea por esa misma aplicación, o por otras distintas a la que originó dicha información.

A lo largo el tiempo han variado los soportes de almacenamiento, los formatos y el modo en el que se organiza la información, y también las tecnologías de acceso a ella. Esta evolución ha sido fruto de los avances en estos tres ejes que, combinados entre sí, han permitido aumentar la capacidad de almacenamiento y la rapidez con la que podemos acceder y persistir la información.

Recordemos que la palabra informática tiene su origen en la palabra francesa "*informatique*", que fue acuñada en 1962 por **Philippe Dreyfus**, que deriva de las palabras informa(tion) y (automa)tique.

Sabías que

Philippe Dreyfus se licenció en Física en 1950 y fue profesor en Harvard, siendo uno de los pioneros de la informática y usando Mark I, el primero ordenador automatizado que existió.

Empezando por las tarjetas perforadas, y evolucionando en cintas magnéticas, unidades de disco duro y flexible, discos ópticos, de estado sólido, etc., los soportes de almacenamiento de información han permitido crecer a la informática, no solamente almacenando datos, sino que también a los programas que permiten generar y tratar de manera automática dicha información, haciendo más fáciles y productivas tareas que hubieran llevado mucho más tiempo hacer, o que sencillamente, hubieran sido imposibles de realizar por el alto coste en tiempo y recursos que implicarían.

Definición

Según el diccionario de la RAE, el término **Informática** se define como:

(Del fr. informatique).

1. f. Conjunto de conocimientos científicos y técnicas que hacen posible el tratamiento automático de la información por medio de ordenadores.

Tech Hub <FP> euroformac

2. Ficheros

Cuando hablamos de ficheros hablamos de una gran variedad de formatos, contenidos, y formas de acceso, no pudiendo incluirlos todos aquí, pero veamos algunos de los más interesantes cuando hablamos de almacenamiento de la información:

Ficheros	Descripción
Planos	Son los más habituales y contienen información en forma de texto, pudiendo estar estructurada o no estructurada. Cualquier fichero que podamos abrir y visualizar con normalidad su contenido desde el bloc de notas, es un fichero plano. Un ejemplo de información no estructurada sería Don Quijote de la Mancha, y un ejemplo de información estructurada, podría ser un fichero con formato CSV, JSON, o XML.
Indexados	Son ficheros que contienen uno o varios índices para facilitar la búsqueda de información dentro del propio fichero. Aunque nacieron antes, convivieron largo tiempo con las bases de datos de hoy en día, y siguen haciéndolo hoy, aunque en menor medida, estos ficheros fueron una muy buena solución para almacenar información que debía recuperarse. Un ejemplo paradigmático de este tipo de ficheros fueron las bases de datos DBASE y algunas de sus derivadas como FOXPRO.
Multimedia	Son ficheros que contienen imágenes, audio y vídeo, y que, en muchas ocasiones, se almacenan dentro de las bases espacios específicos de estas de forma binaria.

2.1 Ficheros planos

Como se ha dicho antes, pueden ser **estructurados** (formato CSV, JSON, XML, etc.) o desestructurados (Don Quijote de la Mancha, El nombre de la rosa, etc.), pudiendo ser consultables y editables desde aplicaciones muy simples como el Bloc de notas, aunque se pueden usar programas más sofisticados como editores de código o entornos de desarrollo, dependiendo del contenido y el formato de los ficheros a tratar.

Para acceder a la información que buscamos, debemos hacer un recorrido secuencial hasta encontrarla, no existiendo información adicional o metainformación con índices que nos permitan acceder directamente a la porción de información que necesitamos.

Pueden intercambiarse fácilmente entre distintas plataformas y sistemas operativos, ya que su contenido no está ligado a estructuras particulares de ninguno de ellos.

Al no disponer de estructuras ajenas al propio contenido, no pueden incorporar mecanismos de seguridad como contraseñas.

Este párrafo del inicio del Quijote es un ejemplo de información desestructurada que pudiera almacenarse en un fichero plano con formato TXT:

> En un lugar de La Mancha, de cuyo nombre
>
> no quiero acordarme, no hace mucho tiempo
>
> que vivía un hidalgo de escudo antiguo, rocín
>
> flaco y galgo corredor.
>
> Llevaba una vida acomodada, aunque sin grandes lujos,
>
> y en su casa nunca faltó comida,
>
> ni ropa con la que vestirse en los días de fiesta.

Otro ejemplo de fichero plano con datos estructurados en formatos CSV, JSON, y XML, es la entidad coche que vimos en el capítulo anterior:

- **Fichero CSV:**

Coches.csv

```
Matrícula;Marca;Modelo;Color;Bastidor
1111-BBB;Seat;Ibiza;Negro;221s123SWse233223
2222-CCC;Opel;Omega;Rojo;83223sww3023230230
3333-DDD;Dacia;Sandero;Azul;03s232323131232323
4444-FFF;Volvo;V40;Negro;30s0332ss11323223
```

- **Fichero JSON:**

.csv | coches.json

```
[
  {
    "Matrícula": "1111-BBB",
    "Marca": "Seat",
    "Modelo": "Ibiza",
    "Color": "Negro",
    "Bastidor": "22ls123SWse233223"
  },
  {
    "Matrícula": "2222-CCC",
    "Marca": "Opel",
    "Modelo": "Omega",
    "Color": "Rojo",
    "Bastidor": "83223sww3023230230"
  },
  {
    "Matrícula": "3333-DDD",
    "Marca": "Dacia",
    "Modelo": "Sandero",
    "Color": "Azul",
    "Bastidor": "03s232323131232323"
  },
  {
    "Matrícula": "4444-FFF",
    "Marca": "Volvo",
    "Modelo": "V40",
    "Color": "Negro",
    "Bastidor": "30s0332ssl1323223"
  }
]
```

- **Fichero XML:**

csv | coches.json | coches.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<root>
  <row>
    <Matrícula>1111-BBB</Matrícula>
    <Marca>Seat</Marca>
    <Modelo>Ibiza</Modelo>
    <Color>Negro</Color>
    <Bastidor>22ls123SWse233223</Bastidor>
  </row>
  <row>
    <Matrícula>2222-CCC</Matrícula>
    <Marca>Opel</Marca>
    <Modelo>Omega</Modelo>
    <Color>Rojo</Color>
    <Bastidor>83223sww3023230230</Bastidor>
  </row>
  <row>
    <Matrícula>3333-DDD</Matrícula>
    <Marca>Dacia</Marca>
    <Modelo>Sandero</Modelo>
    <Color>Azul</Color>
    <Bastidor>03s232323131232323</Bastidor>
  </row>
  <row>
    <Matrícula>4444-FFF</Matrícula>
    <Marca>Volvo</Marca>
    <Modelo>V40</Modelo>
    <Color>Negro</Color>
    <Bastidor>30s0332ssl1323223</Bastidor>
  </row>
</root>
```

Veamos el siguiente ejemplo de forma práctica:

Imagina que trabajas en la tienda online "ModaExpress" como gestionando la parte logística. Cada mañana recibes un fichero llamado inventario.csv con la lista de productos y su stock:

- Producto,Categoría,Stock
- Camiseta Azul,Ropa,12
- Pantalón Vaquero,Ropa,4
- Mochila Escolar,Accesorios,8
- Gorra Verano,Accesorios,2

Tu misión es detectar qué productos tienen stock menor que 5 para pedirlos de inmediato.

Solución paso a paso (con LibreOffice Calc o Excel):

1. Abre el archivo inventario.csv en Calc o Excel.
2. Selecciona la fila de cabecera (Producto, Categoría, Stock).
3. Ve al menú Datos → Filtro automático y activa el filtro.
4. Haz clic en la flecha del filtro de la columna Stock, elige "Número de filtro…" y marca "Menor que 5".
5. Copia las filas resultantes (serán Pantalón Vaquero y Gorra Verano) y pégalas en un nuevo archivo llamado reponer.txt.

Contenido de reponer.txt:

- Pantalón Vaquero,Ropa,4
- Gorra Verano,Accesorios,2

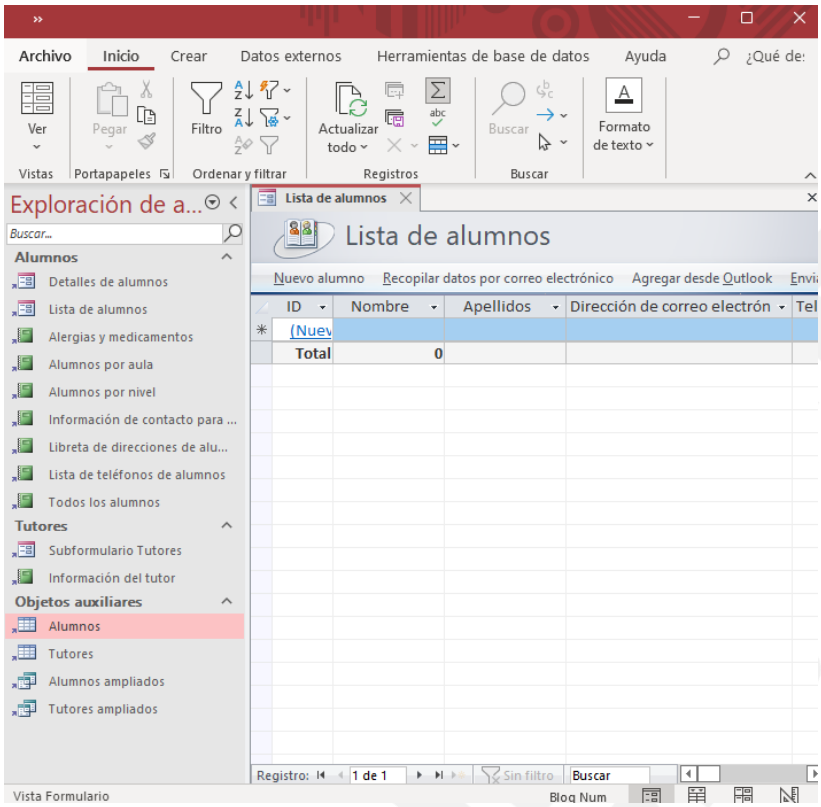
Con este ejercicio aprendes a abrir un fichero plano, aplicar filtros básicos y generar un listado que puedes enviar al proveedor.

2.2 Ficheros indexados

Las principales **características** de los **ficheros indexados** son las siguientes:

Característica	Descripción
Índices	Incorporan uno o varios índices que permiten realizar búsquedas rápidas y eficientes para acceder a la información deseada sin tener que realizar un recorrido secuencial por todo el fichero hasta dar con ella.
Estructura determinada	La información está estructurada de manera que tiene capacidad de buscar aquello que necesita en el sitio en que se encuentra. Entidades, atributos y valores marcan los atajos que puede seguir para encontrar la información deseada.
Masivos	Se utilizan principalmente para almacenar grandes cantidades de información estructurada que debe poderse recuperar en un tiempo razonablemente rápido.
Seguros	Al disponer de metadatos, es posible delimitar el acceso mediante contraseñas y algoritmos de encriptación.
Codificados	Por su naturaleza y la complejidad de las estructuras que pueden albergar, mezclando datos y metadatos de control, son difíciles de editar por programas no especializados. Por ejemplo, las distintas bases de datos que existen actualmente no pueden editarse en un bloc de notas, necesitando de programas especializados en administrarlas, como son los Sistemas Gestores de Bases de Datos (SGBD).

Por ejemplo, podemos comparar como se ve el fichero alumnos.accdb, que es una base de dato ACCESS, desde el programa MS-ACCESS (imagen 1), y como se vería desde el bloc de notas (imagen 2):



Base de datos Access abierta desde MS_ACCES

Base de datos Access abierta desde el bloc de notas

Veamos varios ejemplos que ilustran el uso de índices en distintos formatos de ficheros indexados, lo que permite comprender cómo estos mecanismos mejoran drásticamente el rendimiento de las búsquedas frente a un recorrido secuencial completo. Estos ejemplos cubren tanto formatos clásicos (como dBASE) como sistemas más modernos (SQLite y Access), y muestran cómo se define y se aprovecha un índice en cada uno de ellos:

- dBASE (.dbf)

Imaginamos que desde LibreOffice Base creamos una tabla clientes.dbf con campos ClienteID (entero) y Nombre (cadena). Al definir un índice B-tree sobre ClienteID, cuando buscas el registro con ClienteID = 42 el sistema recorre el árbol en O(log n) en lugar de tener que leer registro a registro los 10 000 clientes.

- SQLite (.sqlite)

Supongamos que gestionamos un inventario en una base inventario.sqlite y creamos un índice B-tree sobre la columna Fecha de la tabla Ventas. Una consulta como
SELECT * FROM Ventas WHERE Fecha BETWEEN '2025-07-01' AND '2025-07-07'
se resuelve en milisegundos porque SQLite navega por el índice en lugar de filtrar todas las filas.

- Access (.accdb)

En el fichero alumnos.accdb de MS-Access (Imagen 3.4) aparece la tabla “Lista de alumnos” con un índice primario en el campo ID. Al hacer clic en el icono Buscar de la cinta de opciones e introducir un valor de ID, Access utiliza automáticamente ese índice para saltar al registro en milisegundos, sin tener que cargar ni recorrer secuencialmente todo el fichero, como ocurriría si lo abriésemos en el Bloc de notas

3. Bases de datos

3.1 Concepto

Una primera definición de **base de datos** que podríamos dar es la siguiente: Una base de datos es un conjunto de información que se encuentra organizada de una determinada manera y que está almacenada en un sistema informático, a la que se puede acceder de manera rápida y eficiente para consultar o actualizar su contenido.

Aunque nos da una idea general de lo que es una base de datos, necesitamos definir algunos conceptos más para conocer mejor cuál es la naturaleza de su contenido. Para ello, definiremos qué es un **atributo**, un **valor**, y una **entidad**.

Definición

Atributo:

Se refiere a una característica concreta de cualquier objeto del que queramos tener información. Para cada atributo se definirá qué tipo de dato puede contener (numérico, alfanumérico, fecha, hora, etc.)

Por ejemplo: la matrícula de un coche, el nombre y los apellidos de una persona, el diámetro de una rueda, la marca de un producto comercial, etc.

Entidad:

Una entidad hace referencia al conjunto de atributos que, siguiendo algún criterio de pertenencia, están relacionados entre sí y los agrupamos bajo el paraguas de esta agrupación de nivel superior a la que llamamos entidad.

Puede haber relaciones entre entidades que se establecerán compartiendo vínculos entre alguno/s de sus atributo/s.

Algunos ejemplos de Entidad pueden ser los siguientes:

Entidad	Atributos que la conforman
Clientes	CIF, razón social, domicilio, CP, etc.
Facturas	Número de la factura, base, IVA, total, número de cliente, etc.
Trabajadores	DNI, nombre, primer y segundo apellido, categoría profesional, etc.
Coches	Número de matrícula, marca, modelo, color, número de bastidor, etc.

Definiendo entidad y atributo, hemos conceptualizado las estructuras organizativas básicas de la base de datos, cuyo cometido es organizar y ordenar la información. Vienen a ser como un molde estructural, donde almacenaremos la información propiamente dicha.

Definición

Valor:

El **valor** será el contenido informativo del que estará dotado un atributo en concreto, y la información que se podrá almacenar en cada atributo vendrá determinada por el tipo de dato que lo define. Habrá tantos valores, como representaciones existan de cada entidad.

Podemos tener una idea visual de cada uno de estos conceptos utilizando una hoja de cálculo para representar la estructura y el contenido de la entidad Coches o Clientes:

	A	B	C	D	E
1	**Matrícula**	**Marca**	**Modelo**	**Color**	**Bastidor**
2	1111-BBB	Seat	Ibiza	Negro	22lsl23SWse233223
3	2222-CCC	Opel	Omega	Rojo	83223sww3023230230
4	3333-DDD	Dacia	Sandero	Azul	03s232323l3l232323
5	4444-FFF	Volvo	V40	Negro	30s0332ssll323223
6					
7					

Clientes | **Coches**

	A	B	C	D	E
1	**CIF**	**Razón Social**	**Domicilio**	**CP**	**Ciudad**
2	A11111111	Sociedad A, S.A.	C/Picasso, 1	28002	Madrid
3	B22222222	Sociedad B, S.L.	C/Balmes, 23	08010	Barcelona
4	A33333333	Sociedad C, S.A.	C/Almansa, 32	43003	Tarragona
5					
6					
7					

Clientes

Si nos fijamos en la hoja de cálculo donde están representados los clientes, vemos que en las columnas se definen los atributos de la entidad, y a partir de la fila 2 se definen los valores de estos atributos. En cada una de estas filas, tendremos representaciones individuales de la entidad Cliente, agrupando los valores relacionados entre sí organizados en atributos.

Una vez definidos Entidad, atributo, y valor, vamos a proporcionar una segunda definición de **base de datos** que complementará a la que dimos al principio de este apartado:

Definición

Base de Datos:

Una base de datos es la representación informática de un conjunto de representaciones de valores organizados en atributos y agrupados en entidades, así como las relaciones que existen entre distintas entidades, que pueden accederse de manera rápida, eficiente, y automatizada para consultar y actualizar su contenido.

3.2 Clasificación de las bases de datos

Hay muchas maneras de clasificar las bases de datos en función del criterio que escojamos. Así pues, podemos clasificarlas según distintos criterios, pudiendo haber muchos más a los enumerados a continuación:

- **Su estructura:** Bases de datos relacionales, no relacionales o documentales, jerárquicas, de red, etc.
- **Su modo de acceso:** Centralizadas o Distribuidas.
- **Su propósito:** Transaccionales u operacionales, de inteligencia de negocio, de inteligencia artificial, etc.
- **Su pertenencia:** Bases de datos personales, empresariales, o gubernamentales.

A continuación, desarrollaremos las clasificadas según su estructura y, en el último epígrafe del tema, las clasificadas según su modo de acceso.

3.3 Tipos de bases de datos según su estructura

Base de Datos	Descripción
Relacionales	Utilizan un esquema de tablas (equivalentes a las entidades que vimos anteriormente), que consta de un conjunto de columnas y filas, en cuya intersección se almacenan los datos, pudiéndose establecer relaciones entre ellas. A filas también se las conoce como tuplas o registros, y a las columnas como atributos o campos. Las relaciones se establecen a través de columnas vinculadas entre dos tablas, de manera que es posible completar la información de una tabla, con información de otras tablas a través de estos vínculos. En temas venideros concretaremos y detallaremos cómo se realiza este proceso. Se utilizan principalmente en aplicaciones empresariales porque su estructura y organización encaja muy bien con los datos y conceptos de índole económica y mercantil. Entre las más utilizadas en la actualidad se encuentran SQL Server, Oracle, MySQL, Maria DB, etc.
NoSQL	También se las conoce como Bases de Datos Documentales. No incorporan la estructura tradicional de tablas relacionales, sino que utilizan distintos modelos de datos, como pueden ser documentos, columnas, grafos y claves-valor. Suelen usarse para almacenar grandes cantidades de datos no estructurados o semiestructurados y son escalables y altamente disponibles. Las redes sociales, suelen recurrir a estas bases de datos para poder almacenar información textual y multimedia, sirviéndola de forma eficiente. Las bases de datos NoSQL más utilizadas actualmente son MongoDB, Azure Cosmos DB, Cassandra, Redis, Neo4j, etc.
Orientadas a Objetos	Las Bases de Datos orientadas a Objetos permiten describir objetos del mundo real, como se hace en lenguajes de programación que utilizan este paradigma (Java, C#, C++, etc.), superando algunas de las reglas estrictas que suponen las bases de datos relacionales. Entre las ventajas de este tipo de bases de datos respecto a las relacionales están: - Modelo de objetos más próximo al mundo real y que permiten encajar mejor aplicaciones y bases de datos, sin tener que recurrir a estructuras de transferencia entre ambos entornos. - Se pueden aprovechar mecanismos como la herencia. - Las consultas sobre los datos son más expresivas y flexibles.

	Entre sus desventajas están la mayor complejidad de su diseño, y por tanto, menor personal técnico especializado disponible, así como limitaciones en las consultas producidas por una de las características del paradigma de la programación orientada a objetos, la encapsulación.
Jerárquicas	En este tipo, los datos están organizados en una estructura jerárquica parecida a un árbol, con ramas, subramas y hojas. Cada nivel de la jerarquía es un conjunto de datos relacionado y los datos de un determinado nivel se relacionan con los datos del nivel superior o inferior. También se utilizan en sistemas empresariales en los que los datos se pueden organizar en forma de árbol, como por ejemplo productos categorizados, esquemas orgánicos de recursos humanos, etc.
De Red	Los datos están divididos en registros, y cada registro se puede relacionar con muchos otros registros a través de las relaciones que se definan. Esto confiere mayor flexibilidad en la forma de organizar los datos y en la representación de relaciones complejas entre ellos. Esta estructuración de los datos es adecuada para sistemas en los que los datos están relacionados de manera no jerárquica o en varios niveles de relaciones, como puede ocurrir en aplicaciones de redes sociales etc.

Ejemplos sobre la elección de la estructura adecuada en diferentes situaciones

Caso práctico resuelto 1 – Sistema de facturación empresarial

Contextualización:
Imagina que trabajas en el departamento de TI de una empresa de distribución que factura miles de pedidos cada mes. Cada factura debe referenciarse al cliente correspondiente, a los productos vendidos y al estado de pago, y cualquier error en la relación entre tablas podría producir desequilibrios contables.
¿Qué tipo de base de datos debes usar para garantizar la integridad de datos y soportar transacciones seguras entre facturas, clientes y pagos?

Solución:
Se debe utilizar una base de datos relacional. Las tablas permiten definir claves primarias y foráneas que aseguran la integridad referencial entre facturas, clientes y pagos. Además, los SGBD relacionales ofrecen transacciones ACID para que operaciones como “crear factura” y “registrar

pago" sean atómicas y consistentes, evitando pérdidas o duplicados de información.

Caso práctico resuelto 2 – Almacenamiento de metadatos de vídeo

Contextualización:
Trabajas en una startup de streaming de vídeo que almacena millones de documentos JSON con metadatos (título, descripción, etiquetas, duración) y necesita escalar rápido cuando aumentan los usuarios y el catálogo. ¿Qué tipo de base de datos es más adecuado para manejar grandes volúmenes de documentos semiestructurados y permitir crecimiento horizontal eficiente?

Solución:
La opción recomendada es una base de datos NoSQL documental, como MongoDB. Maneja de forma nativa estructuras JSON, no requiere esquema fijo y escala fácilmente añadiendo nodos al clúster. Así, las consultas y escrituras de metadatos son rápidas y el sistema crece al ritmo del negocio sin complejos rediseños de tablas.

Caso práctico resuelto 3 – Gestión de objetos complejos en desarrollo

Contextualización:
Formas parte de un equipo que desarrolla una aplicación en Java para plantas de automatización industrial. Modelas piezas y mecanismos con herencia, métodos y propiedades, y quieres guardar esos objetos directamente sin convertirlos en filas y columnas.

¿Qué tipo de base de datos te permite almacenar y recuperar los objetos Java con su jerarquía e información de manera directa?

Solución:
Debes usar una base de datos orientada a objetos. Este modelo almacena las clases, sus atributos y su herencia tal cual existen en el código, evitando el "impedance mismatch" y simplificando el persistir y recuperar objetos con sus métodos y relaciones sin transformarlos manualmente en registros relacionales.

Caso práctico resuelto 4 – Organigrama de un hospital

Contextualización:
Trabajas en el departamento de sistemas de un hospital y mantienes la estructura de departamentos, unidades y subunidades. Necesitas consultar rápidamente todos los niveles superiores o inferiores de cualquier sección del organigrama.

¿Qué modelo de base de datos facilita consultar ramas enteras de un árbol jerárquico sin complejas uniones?

Solución:
Se elige una base de datos jerárquica. Su estructura de árbol nativo permite navegar fácilmente de padre a hijos y viceversa, consultando en un solo recorrido todo el subárbol de una sección o los ancestros de un nodo, ideal para organigramas rígidos y profundos.

Caso práctico resuelto 5 – Red de abonados y centrales

Contextualización:
Eres analista en una compañía telefónica donde cada abonado puede conectarse a varias centrales y cada central dar servicio a muchos abonados. Necesitas recorrer estas conexiones en ambas direcciones para detectar puntos de congestión.

¿Qué tipo de base de datos permite modelar y navegar relaciones muchos-a-muchos de forma directa y eficiente?

Solución:
La base de datos de red es la más adecuada. Permite definir relaciones directas muchos-a-muchos entre registros (abonado ↔ central) y facilita la navegación bidireccional sin tablas intermedias, ofreciendo rutas de acceso eficientes para explorar todas las conexiones dentro de la red.

4. Sistemas Gestores de Bases de Datos

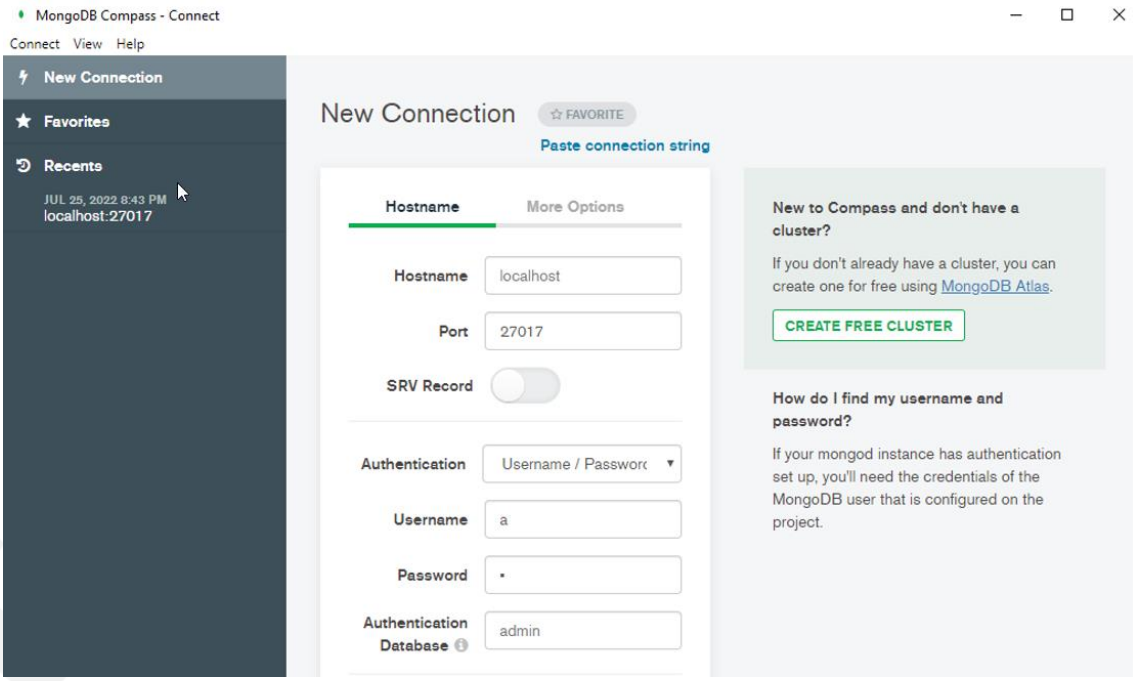
4.1 Concepto

Podemos definir los **Sistemas Gestores de Bases de Datos** (**SGBD**), como aplicaciones que nos permiten administrar y gestionar nuestras bases de datos.

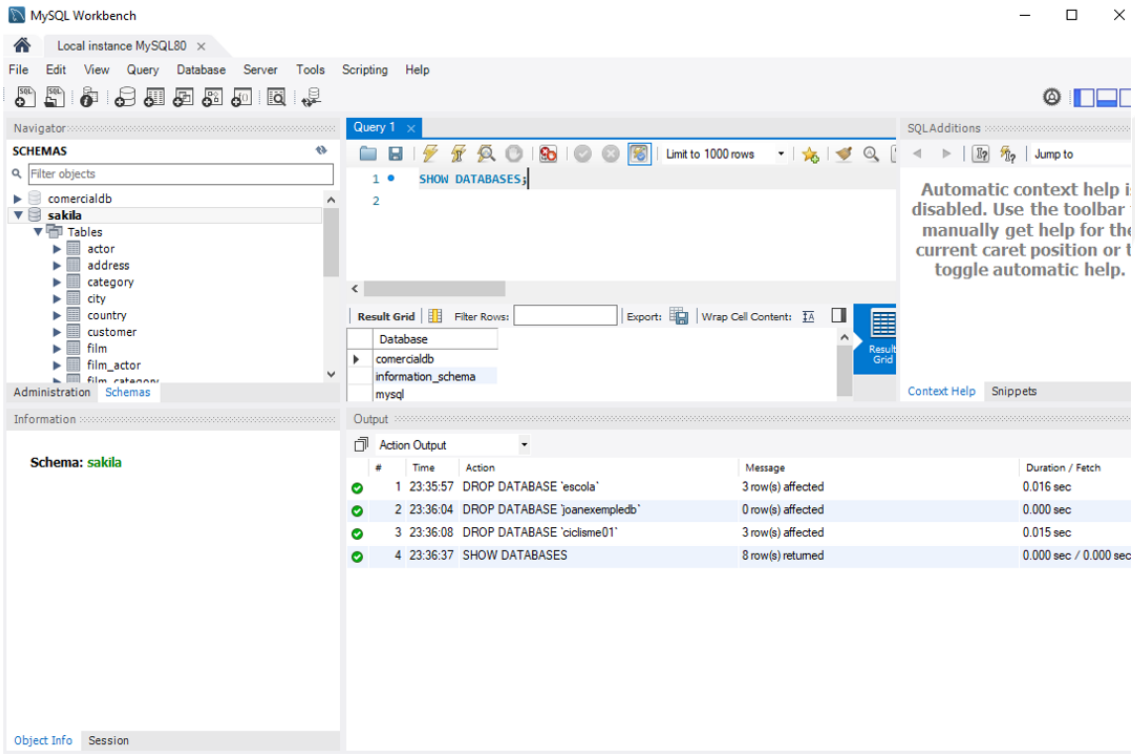
4.2 Tipos de SGBD

Existen tantos tipos de SGBD como tipos de bases de datos hay:

- Relacionales.
- NoSQL.
- Jerárquicos.
- De red.
- Etc.



SGBD MongoDB Compass



SGBD MySQL Workbench

4.3 Objetivos y funcionalidades de los SGBD

Entre los principales **objetivos** y **funcionalidades** que deben incorporar estos aplicativos, se encuentran:

- Definir y asegurar los datos.
- Permitir su recuperación en caso de desastres.
- Proteger la integridad de los datos respetando las restricciones establecidas en los diseños de las bases de datos.
- Garantizar la independencia lógica y física de los datos.
- Permitir la concurrencia de usuarios.
- Etc.

4.4 Componentes de los SGBD

Todo SGBD está formado en dos grandes componentes que, a su vez, pueden dividirse en otros subcomponentes. Entre los más comunes, destacaremos:

- **Gestor de almacenamiento que está compuesto por:**
  - Gestor de autorizaciones e integridad.
  - Gestor de transacciones.
  - Gestor de ficheros.
  - Gestor de memoria intermedia.
- **Procesador de consultas:**
  - Intérprete DDL: Interpreta las sentencias de creación de estructuras como tablas y atributos, relaciones, etc.
  - Intérprete DML: Interpreta las sentencias de consulta y manipulación de datos (altas, cambios y eliminación).

Sabías que

SQL (Structured Query Language) es el lenguaje que utilizan los SGBD relacionales para codificar las instrucciones que permiten crear estructuras, manipular datos, gestionar usuarios y permisos, etc.

Aunque no está plenamente estandarizado, tiene muchos puntos en común en los distintos SGBD, por lo que sentencias DDL y DML suelen ser muy parecidas o idénticas en todos ellos.

Veamos el siguiente ejemplo práctico:

Imagina que eres el responsable de la biblioteca de tu barrio, que ha decidido informatizarse con un SGBD para gestionar el préstamo de libros, el catálogo y las altas de nuevos socios. Veamos cómo entran en juego sus dos grandes "módulos":

1. **Gestor de almacenamiento**

Es la parte encargada de guardar, **recuperar y proteger** físicamente los datos (libros, socios, préstamos). Dentro de ella hay:

a. **Gestor de autorizaciones e integridad**

Ejemplo: solo el bibliotecario puede dar de alta nuevos usuarios o eliminar libros obsoletos; además, asegura que no puedas prestar dos veces el mismo **ejemplar si ya está en préstamo.**

b. **Gestor de transacciones**

Ejemplo: el acto de "prestar un libro" agrupa varias acciones (marcar el libro como prestado, apuntar la fecha de devolución y notificar al socio). Si algo falla (p.ej. se cae la red), el gestor de transacciones "deshace" todo para no dejar la base de datos en estado inconsistente.

c. **Gestor de ficheros**

Ejemplo: organiza en disco los archivos que contienen las tablas "Libros", "Socios" y "Préstamos", decidiendo en qué posición física va cada registro.

d. **Gestor de memoria intermedia (buffer)**

Ejemplo: cuando consultas los datos del libro "Código Da Vinci", el gestor carga la página con su información en memoria RAM para que las siguientes lecturas o escrituras sean muy rápidas, sin volver a leer del disco hasta que sea necesario.

2. **Procesador de consultas**

Se encarga de **entender y ejecutar** las órdenes que tú le das en SQL:

a. **Intérprete DDL (Data Definition Language)**

- Ejemplo: si tú ordenas

```
CREATE TABLE Socios (
  SocioID INT PRIMARY KEY,
  Nombre VARCHAR(50),
  FechaAlta DATE
);
```

El intérprete DDL traduce esa sentencia y crea en disco la estructura física de la tabla “Socios”.

b. Intérprete DML (Data Manipulation Language)

- Ejemplo: para registrar un nuevo préstamo escribes

INSERT INTO Préstamos VALUES (1001, 42, '2025-07-05');

y para ver los libros en préstamo haces

SELECT LibroID, SocioID FROM Préstamos WHERE FechaDevolución IS NULL;

El intérprete DML analiza y envía estos comandos al gestor de almacenamiento para que los aplique sobre los ficheros.

Gracias a esta división en módulos, el SGBD puede optimizar cada tarea (protección, rendimiento, análisis de sentencias) de forma independiente, garantizando fiabilidad y velocidad en tu “biblioteca digital”.

5. SGBD Comerciales vs SGBD Libres

Los Sistemas de Gestión de Bases de Datos comerciales y libres son diferentes en términos de costo, flexibilidad y control de la propiedad.

Los DBMS comerciales suelen ser costosos y tienen licencias de uso restringidas, pero ofrecen soporte técnico y garantía de calidad. Estos son desarrollados y vendidos por compañías especializadas en tecnología de la información, y suelen incluir características avanzadas y herramientas de gestión.

Por otro lado, los DBMS libres son de código abierto y no tienen costo de licencia, lo que los hace más accesibles. Sin embargo, estos pueden requerir habilidades técnicas avanzadas para su uso y configuración, y el soporte técnico puede ser limitado o depender de la comunidad de usuarios.

En resumen, la elección entre un DBMS comercial o libre depende de las necesidades y recursos de cada organización.

La elección de un sistema gestor de bases de datos debe fundamentarse en varios criterios clave que reflejan tanto las necesidades técnicas como los recursos disponibles:

- Modelo de datos y funcionalidad. Se debe evaluar si la aplicación exige un esquema fijo de tablas y relaciones (modelo relacional), o bien requiere flexibilidad para documentos, grafos o pares clave–valor (modelos NoSQL). También conviene valorar la necesidad de soporte para transacciones complejas, herencia de objetos o replicación distribuida.
- Volumen de datos y escalabilidad. Resulta imprescindible prever el crecimiento de los datos: en sistemas que escalan verticalmente, puede bastar un servidor potente; en escenarios de expansión horizontal, es preferible un SGBD con capacidades de particionado (sharding) y replicación nativa.
- Rendimiento y consistencia. La concurrencia de usuarios y los requisitos de consistencia (ACID frente a consistencia eventual) influyen directamente en el rendimiento. En entornos críticos de escritura y lectura simultáneas, un SGBD con optimización de índices y gestión avanzada de bloqueos ofrecerá menores tiempos de respuesta.
- Coste y modelo de licenciamiento. Los SGBD comerciales implican gastos en licencias, mantenimiento y soporte, mientras que las soluciones libres suprimen el coste de licencia pero pueden requerir inversión en formación y soporte externo. Es clave comparar el coste total de propiedad a medio y largo plazo.

- Soporte, comunidad y ecosistema. Un producto con soporte oficial aporta garantías de nivel de servicio (SLA) y actualizaciones planificadas. En cambio, las opciones de código abierto dependen de la actividad de su comunidad, la calidad de la documentación disponible y el ecosistema de herramientas de administración y desarrollo.
- Seguridad y cumplimiento normativo. Es fundamental revisar las capacidades de autenticación, cifrado (en reposo y en tránsito), control de acceso granular y auditoría. El SGBD debe facilitar el cumplimiento de normativas como GDPR, ISO 27001 o PCI-DSS cuando se manejen datos sensibles.
- Operaciones de mantenimiento. La sencillez de tareas diarias —copias de seguridad, recuperación ante desastres, monitorización de rendimiento— condiciona el perfil del personal necesario. Algunos SGBD ofrecen asistentes gráficos y automatización para reducir la carga administrativa.
- Riesgo de dependencia y futuro de la solución. Finalmente, conviene valorar la hoja de ruta del producto, la frecuencia de sus actualizaciones y el riesgo de "vendor lock-in". Una buena interoperabilidad y la existencia de herramientas de migración facilitan futuros cambios de plataforma sin afectar la continuidad del servicio.

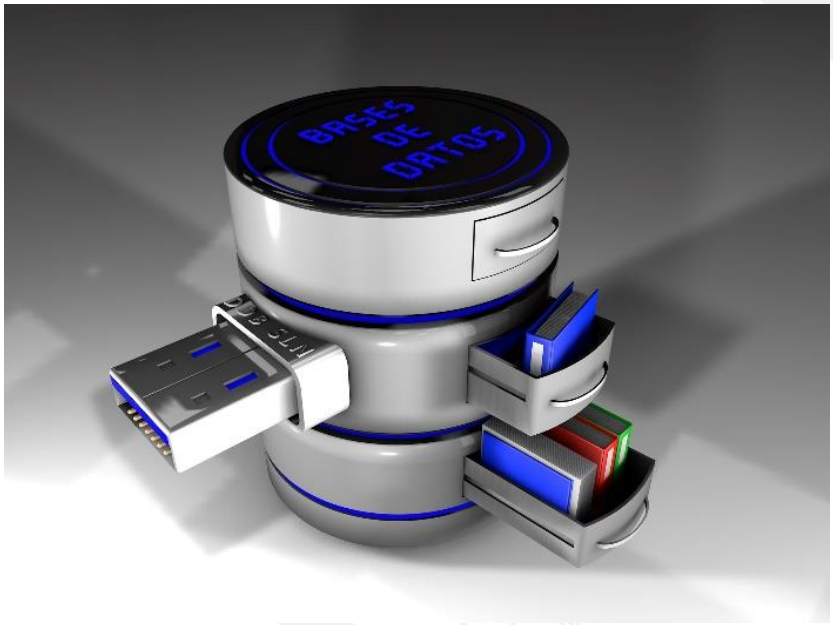
5.1 SGBD Comerciales

Las **características** principales de los Sistemas de Gestión de Bases de Datos comerciales incluyen:

- **Costo**: Los DBMS comerciales suelen tener un costo de licencia y pueden requerir contratos de mantenimiento y soporte técnico.
- **Soporte técnico**: Estos sistemas suelen contar con un equipo de soporte técnico dedicado para resolver problemas y brindar asistencia.
- **Herramientas de gestión:** Los DBMS comerciales suelen incluir herramientas de gestión y monitoreo avanzadas para optimizar la performance y la seguridad de la base de datos.
- **Características avanzadas:** Los DBMS comerciales suelen tener características avanzadas como replicación de datos, recuperación ante desastres y escalabilidad.
- **Control de propiedad**: Los DBMS comerciales son propiedad de la compañía que los desarrolla y vende, lo que les da el control sobre las actualizaciones y mejoras del producto.

Algunas de las marcas de Sistemas de Gestión de Bases de Datos (SGBD) comerciales más conocidas son:

- Oracle.
- Microsoft SQL Server.
- DB2.



5.2 SGBD Libres

Los Sistemas de Gestión de Bases de Datos Libres (SGBD) suelen tener las siguientes características:

- **Código abierto:** Permite a los desarrolladores y administradores de bases de datos acceder al código fuente y personalizarlo según sus necesidades.
- **Costo cero**: Los SGBD libres son gratuitos y no requieren licencias o pagos recurrentes.
- **Comunidad**: Existe una comunidad global de desarrolladores, administradores de bases de datos y usuarios que colaboran en el desarrollo y mejora de la solución.
- **Flexibilidad**: Ofrecen una amplia variedad de opciones de configuración y personalización.
- **Compatibilidad**: Se integran con una amplia variedad de sistemas y aplicaciones.

- **Documentación**: Suelen tener una amplia documentación y recursos en línea disponibles para los usuarios.
- **Funcionalidad**: Ofrecen una amplia gama de funciones, desde el almacenamiento y recuperación de datos hasta el análisis y la visualización.
- **Escalabilidad**: Pueden manejar grandes cantidades de datos y usuarios concurrentes.

Importante

Es importante destacar que, aunque los SGBD libres pueden ser una opción atractiva debido a su costo cero, pueden requerir más esfuerzo y conocimiento técnico para implementarlos y mantenerlos en comparación con los SGBD comerciales. Además, la disponibilidad de soporte y recursos de la comunidad puede variar según la solución elegida.

Algunos de las marcas de Sistemas de Gestión de Bases de Datos (SGBD) libres más conocidas son:

- MySQL.
- PostgreSQL.
- MariaDB.
- MongoDB.
- Apache Cassandra.

6. Bases de datos centralizadas y distribuidas

6.1 Bases de datos centralizadas

Todos los datos se almacenan en un solo lugar, generalmente en un servidor central.

El acceso a los datos se realiza a través de una red desde diferentes lugares o máquinas. Los datos son administrados y controlados por un equipo central de administradores de bases de datos, quienes aseguran la integridad, seguridad y disponibilidad de los datos.

Este tipo de estructura de datos es adecuado para sistemas en los que se requiere un control centralizado sobre los datos y una única fuente de verdad, siendo muy comunes en sistemas empresariales, aunque organizaciones muy grandes como gobiernos o bancos, pueden recurrir al uso de bases de datos distribuidas para asegurar la información y servirla eficientemente en tiempo y forma.

6.2 Bases de datos distribuidas

Los datos se almacenan en múltiples sitios o dispositivos conectados a través de una red.

Cada sitio o dispositivo contiene una parte de los datos y se comunica con los demás sitios para garantizar la consistencia y la disponibilidad de los datos. Es posible tener una o varias copias de los datos en los diferentes sitios.

Estos sistemas son ideales para aplicaciones que requieren escalabilidad, disponibilidad, y tolerancia a fallos.

Al distribuir los datos, se pueden reducir los cuellos de botella, mejorar la seguridad y aumentar la disponibilidad de los datos.

El acceso a los datos se realiza a través de un sistema de control de transacciones distribuido para garantizar la integridad de los datos.

Este tipo de bases de datos son comunes en aplicaciones empresariales y de negocios, especialmente en sistemas de alto rendimiento, como las redes sociales, los sistemas de *e-commerce*, y sistemas de servicios en la nube.

7. Resumen

El **almacenamiento** en bases de datos es un componente clave en la gestión de datos y se utiliza para guardar, organizar y recuperar información.

Existen diferentes tipos de almacenamiento, como discos duros, discos de estado sólido (SSD), discos de sistema de archivos, bases de datos relacionales y NoSQL, y cada uno tiene sus propias características y usos específicos.

El rendimiento, la escalabilidad, la disponibilidad y la seguridad son algunos de los factores críticos que deben tenerse en cuenta a la hora de elegir un sistema de almacenamiento.

Tech Hub <FP> euroformac

Glosario

Termino	Definición
Base de datos	SQL (Structured Query Language) es un lenguaje de programación utilizado para interactuar con bases de datos relacionales. Permite a los usuarios insertar, actualizar, eliminar y recuperar datos de una base de datos, así como crear y modificar estructuras de tablas y restricciones de integridad.
Bases de datos Centralizadas	Se encuentran en un solo lugar y son accedidas por todos los usuarios y aplicaciones a través de una red. Todos los datos se almacenan en un servidor central, permitiendo un control centralizado sobre la información y una gestión más eficiente.
Bases de datos Distribuidas	Se encuentran en múltiples lugares y son accedidas a través de una red. Cada parte de la base de datos se almacena en un servidor o nodo diferente, lo que permite una mayor escalabilidad y disponibilidad que las bases de datos centralizadas.
Bases de datos Jerárquicas	Utilizan un modelo de datos jerárquico para representar la información. En este modelo, los datos se organizan en forma de árbol, con nodos y ramas que representan la información y sus relaciones.
Bases de datos NoSQL	Es un tipo de base de datos que utiliza una variedad de modelos de datos, como documentos, clave-valor, grafos, columnas, etc., permitiéndole manejar grandes cantidades de datos de manera más flexible y escalable que las bases de datos relacionales.
Bases de datos Orientadas a objetos	Utilizan un modelo de datos basado en objetos y clases para representar la información. Este modelo se asemeja a la programación orientada a objetos y permite una representación más flexible y natural de la información que los modelos de datos relacionales y NoSQL.
Bases de datos de Red	Utilizan un modelo de datos basado en redes para representar la información. Los datos se organizan en forma de nodos y relaciones, lo que permite una representación más flexible de la información que los modelos de datos relacionales y jerárquicos.
Bases de datos relacionales	Es un tipo de base de datos que utiliza un modelo de datos basado en relaciones entre tablas para almacenar y acceder a la información. Cada tabla representa una entidad o un objeto, y las relaciones entre las tablas se establecen mediante claves foráneas que vinculan registros en diferentes tablas.
Fichero Indexado	Es un tipo de fichero que utiliza un índice para facilitar la búsqueda y recuperación de registros. El índice es una estructura de datos que almacena información clave sobre

	los registros, como la posición en el archivo y el contenido de los campos clave.
Fichero plano	Un fichero plano es un tipo de fichero de texto sin formato que contiene registros separados por un delimitador, como una coma (CSV) o una tabulación (TSV). Cada registro representa una fila en una tabla y cada campo se separa por el delimitador correspondiente, y a menudo se utilizan para transferir datos entre aplicaciones y sistemas que no son compatibles entre sí.
Sistema Gestor de Bases de datos	Un SGBD (DBMS, por sus siglas en inglés) es un software que proporciona una interfaz entre los usuarios y las bases de datos, permitiendo a los usuarios acceder, almacenar, modificar, buscar y gestionar la información de la base de datos de manera eficiente y segura. También se encarga de la gestión interna de la base de datos, incluyendo el almacenamiento de datos, la gestión de transacciones, la recuperación de datos en caso de fallos, la optimización de consultas, entre otras tareas.

Bibliografía

Navarro, J. (2009). Bases de datos relacionales. Anaya Multimedia.

Suárez, M. (2011). Sistemas de gestión de bases de datos. Pearson.

Díaz, J. (2010). Bases de datos distribuidas. Ra-Ma.

Tech Hub <FP> euroformac